Project 3: Probabilistic Optimization

1 Portfolio Layout Optimization

2

```
2,8.00000000
3,6.92820323
4,5.65685425
5,4.70228202
6,4.00000000
7,3.34926665
8,3.05529777
9,2.74391668
10,2.69575702
```

3

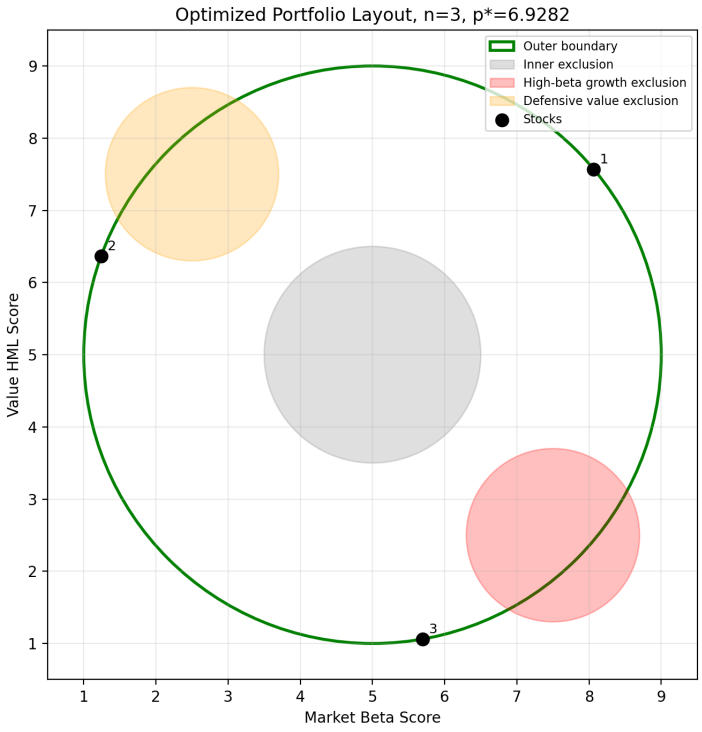

Figure 1: (a) $n = 3$

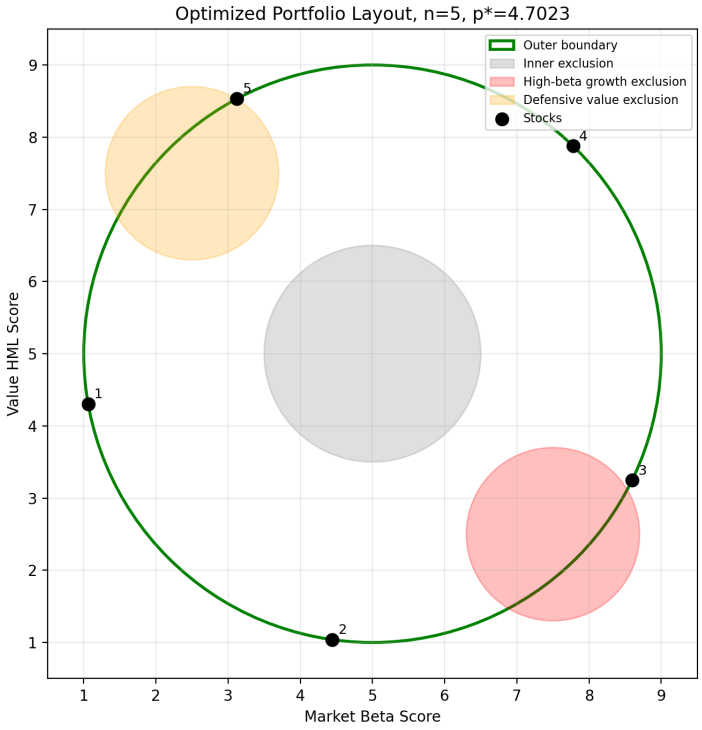

Figure 2: (b) $n = 5$

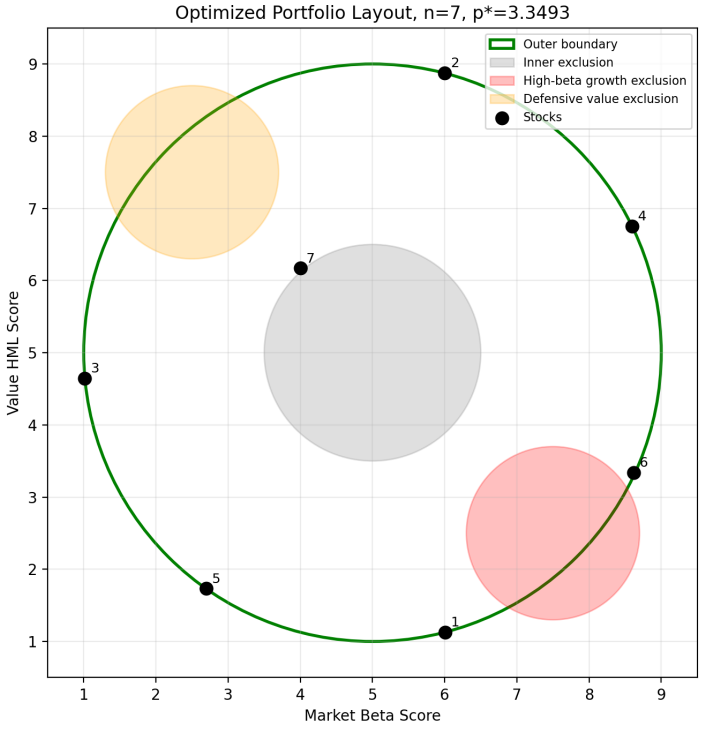

Figure 3: (c) $n = 7$

4

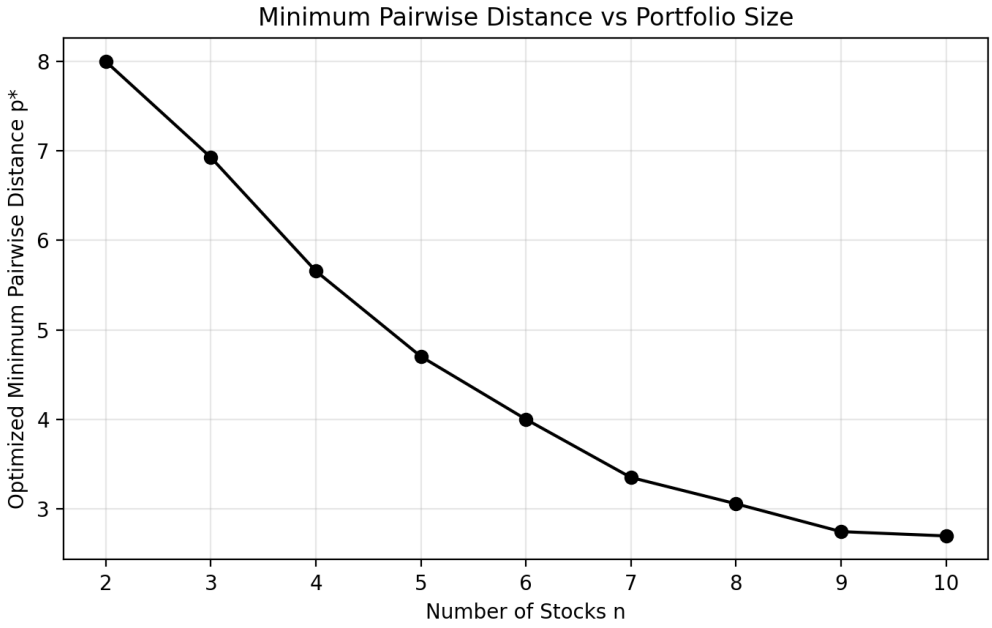

Figure 4: Minimum pairwise distance $p^\star$ versus number of stocks.

5

Figure 5: Theoretical diversification efficiency as a function of $p^\star$

Code

The implementation is included in Appendix A.1.

2 Gaussian Process Fitting

1

The implementation is included in Appendix A.2

2

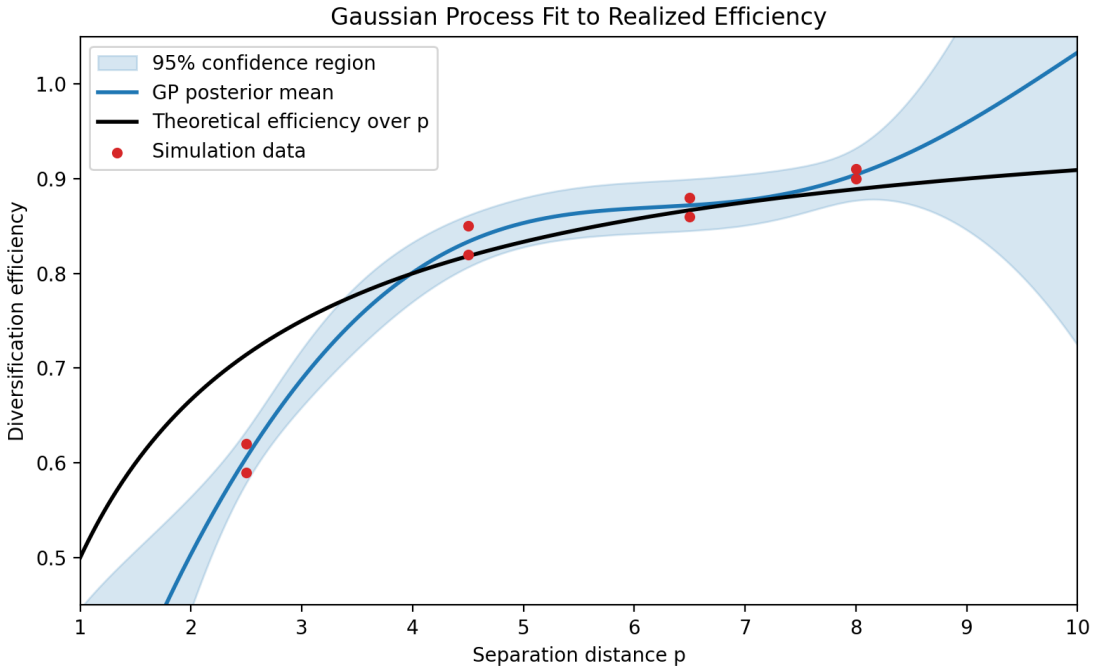

Figure 6: Gaussian process fit for realized diversification efficiency.

3 Optimization Under Uncertainty

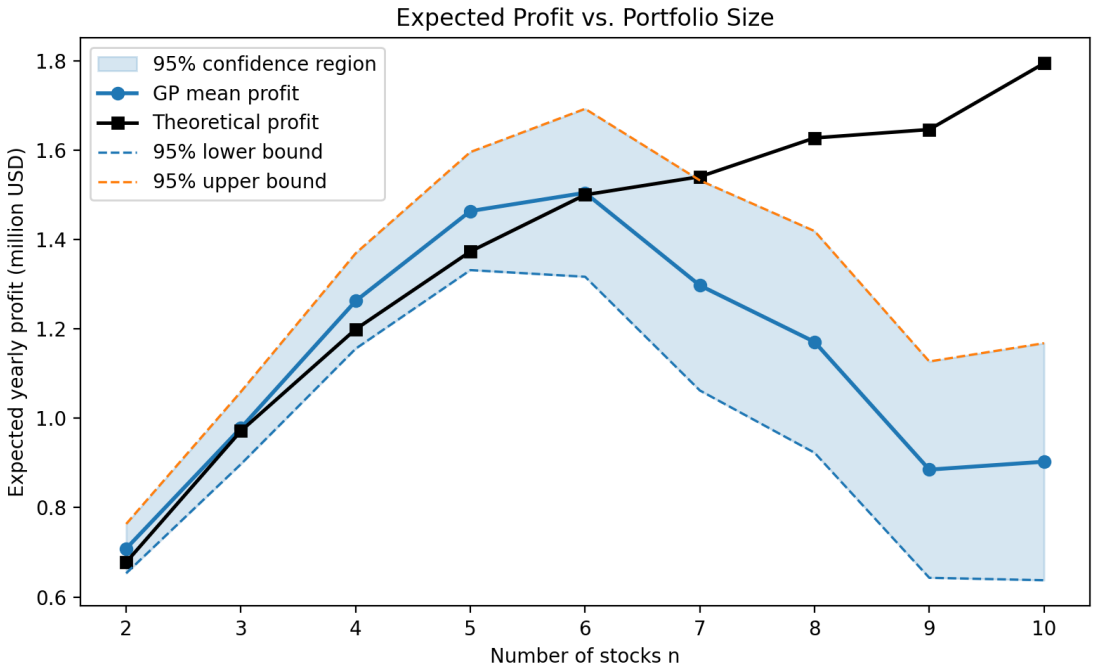

Figure 7: Expected yearly profit as a function of portfolio size.

Recommendation

The recommended portfolio size is $n = 6$. Using only the theoretical diversification curve, the largest expected yearly profit occurs at $n = 10$, because adding more stocks increases the gross return term even though the minimum separation distance decreases. However, the Gaussian-process model learned from the simulation data gives a more conservative efficiency estimate at small separations. Under the GP posterior mean, the best expected profit is at $n = 6$, about \$1.50M per year.

The uncertainty bounds support the same general decision. The lower 95% confidence bound is maximized at $n = 5$, while the upper 95% confidence bound is maximized at $n = 6$. This means the main tradeoff is between $n = 5$ and $n = 6$: $n = 5$ is slightly more conservative, but $n = 6$ has the best posterior-mean and optimistic performance. Therefore, I would choose $n = 6$ unless the manager is strongly risk averse, in which case $n = 5$ is the safer alternative. Additional simulation effort should focus near the corresponding distances $p^\star \approx 4.0$ and $p^\star \approx 4.7$, where the decision is most sensitive to uncertainty.

Code

The implementation is included in Appendix A.3.

A Code

A.1 Portfolio Layout Optimization

```
ZONE_1 = np.array([5.0, 5.0])
ZONE_2 = np.array([7.5, 2.5])
ZONE_3 = np.array([2.5, 7.5])
INNER_RADIUS = 1.5
OUTER_RADIUS = 4.0
EXCLUSION_RADIUS = 1.2

def feasible_points(points):
    d1 = np.linalg.norm(points - ZONE_1, axis=1)
    d2 = np.linalg.norm(points - ZONE_2, axis=1)
    d3 = np.linalg.norm(points - ZONE_3, axis=1)
    return (
        (d1 >= INNER_RADIUS)
        & (d1 <= OUTER_RADIUS)
        & (d2 >= EXCLUSION_RADIUS)
        & (d3 >= EXCLUSION_RADIUS)
    )

def min_pairwise_distance(x, n):
    points = x.reshape(n, 2)
    i, j = np.triu_indices(n, k=1)
    return np.min(np.linalg.norm(points[i] - points[j], axis=1))

def layout_score(x, n):
    points = x.reshape(n, 2)
    if not np.all(feasible_points(points)):
        return -np.inf
    return min_pairwise_distance(x, n)

def project_point(point, n_passes=5):
    point = point.copy()
    for _ in range(n_passes):
        for center, lower, upper in (
            (ZONE_1, INNER_RADIUS, OUTER_RADIUS),
            (ZONE_2, EXCLUSION_RADIUS, np.inf),
            (ZONE_3, EXCLUSION_RADIUS, np.inf),
        ):
            direction = point - center
            distance = np.linalg.norm(direction)
            if distance < 1e-12:
                direction = np.array([1.0, 0.0])
                distance = 1.0
            if distance < lower:
                point = center + direction / distance * lower
            elif distance > upper:
                point = center + direction / distance * upper
    return point

def project_layout(x, n):
    points = x.reshape(n, 2).copy()
```

```
    valid = feasible_points(points)
    for i in np.where(~valid)[0]:
        points[i] = project_point(points[i])
    return points.reshape(2 * n)

def boundary_candidate_points(rng):
    candidates = np.vstack([
        circle_points(ZONE_1, OUTER_RADIUS, 1000),
        circle_points(ZONE_1, INNER_RADIUS, 1000),
        annulus_ring_points(n_rings=6, points_per_ring=240),
        circle_points(ZONE_2, EXCLUSION_RADIUS, 1000),
        circle_points(ZONE_3, EXCLUSION_RADIUS, 1000),
        sample_feasible_points(rng, 5000),
    ])
    return candidates[feasible_points(candidates)]

def greedy_maxmin_layout(candidates, n, start_idx):
    selected = [candidates[start_idx]]
    available = np.ones(len(candidates), dtype=bool)
    available[start_idx] = False
    nearest_dist = np.linalg.norm(candidates - selected[0], axis=1)

    while len(selected) < n:
        masked_dist = np.where(available, nearest_dist, -np.inf)
        next_idx = int(np.argmax(masked_dist))
        selected.append(candidates[next_idx])
        available[next_idx] = False
        new_dist = np.linalg.norm(candidates - candidates[next_idx], axis
            =1)
        nearest_dist = np.minimum(nearest_dist, new_dist)

    return np.array(selected).reshape(2 * n)

def greedy_candidate_layouts(rng, n, n_starts, interior_anchor_fraction
    =0.8):
    candidates = boundary_candidate_points(rng)
    interior = sample_feasible_points(rng, max(4 * n_starts, 1000))
    interior_start = len(candidates)
    candidates = np.vstack([candidates, interior])

    anchors = [
        int(np.argmax(candidates[:, 0])),
        int(np.argmin(candidates[:, 0])),
        int(np.argmax(candidates[:, 1])),
        int(np.argmin(candidates[:, 1])),
    ]
    n_extra = max(0, n_starts - len(anchors))
    n_interior = int(np.ceil(interior_anchor_fraction * n_extra))
    n_global = n_extra - n_interior
    interior_idx = np.arange(interior_start, len(candidates))

    if n_interior:
        anchors.extend(rng.choice(interior_idx, size=n_interior, replace=
            False))
```

```
    if n_global:
        anchors.extend(rng.choice(len(candidates), size=n_global, replace=False))

    return [
        greedy_maxmin_layout(candidates, n, int(idx))
        for idx in anchors[:n_starts]
    ]

def cma_es(f, n, k_max, m, m_elite, rng, sigma0=2.0, x0=None):
    dim = 2 * n
    weights = np.log(m_elite + 0.5) - np.log(np.arange(1, m_elite + 1))
    weights = weights / np.sum(weights)
    mu_eff = 1.0 / np.sum(weights ** 2)

    c_sigma = (mu_eff + 2.0) / (dim + mu_eff + 5.0)
    d_sigma = 1.0 + 2.0 * max(
        0.0, np.sqrt((mu_eff - 1.0) / (dim + 1.0)) - 1.0
    ) + c_sigma
    c_c = (4.0 + mu_eff / dim) / (dim + 4.0 + 2.0 * mu_eff / dim)
    c1 = 2.0 / ((dim + 1.3) ** 2 + mu_eff)
    c_mu = min(
        1.0 - c1,
        2.0 * (mu_eff - 2.0 + 1.0 / mu_eff) / ((dim + 2.0) ** 2 + mu_eff),
    )

    mean = random_feasible_layout(rng, n) if x0 is None else x0.copy()
    sigma = sigma0
    C = np.eye(dim)
    p_c = np.zeros(dim)
    p_sigma = np.zeros(dim)
    expected_norm = np.sqrt(dim) * (1.0 - 1.0 / (4.0 * dim) + 1.0 / (21.0 * dim ** 2))
    best_x = mean.copy()
    best_score = f(best_x)

    for generation in range(k_max):
        eigenvalues, B = np.linalg.eigh(0.5 * (C + C.T))
        eigenvalues = np.maximum(eigenvalues, 1e-12)
        C_sqrt = B @ np.diag(np.sqrt(eigenvalues))
        C_inv_sqrt = B @ np.diag(1.0 / np.sqrt(eigenvalues)) @ B.T

        samples = mean + sigma * (rng.standard_normal((m, dim)) @ C_sqrt.T)
        samples = np.array([project_layout(sample, n) for sample in samples])
        scores = np.array([f(sample) for sample in samples])

        i_best = np.argmax(scores)
        if scores[i_best] > best_score:
            best_score = scores[i_best]
            best_x = samples[i_best].copy()

        elite = samples[np.argsort(scores)[-m_elite:][::-1]]
```

```
        old_mean = mean.copy()
        mean = weights @ elite
        y_w = (mean - old_mean) / sigma

        p_sigma = (
            (1.0 - c_sigma) * p_sigma
            + np.sqrt(c_sigma * (2.0 - c_sigma) * mu_eff) * (C_inv_sqrt @ y_w)
        )
        norm_p_sigma = np.linalg.norm(p_sigma)
        h_sigma = norm_p_sigma / np.sqrt(
            1.0 - (1.0 - c_sigma) ** (2.0 * (generation + 1))
        ) < (1.4 + 2.0 / (dim + 1.0)) * expected_norm
        p_c = (
            (1.0 - c_c) * p_c
            + h_sigma * np.sqrt(c_c * (2.0 - c_c) * mu_eff) * y_w
        )

        elite_y = (elite - old_mean) / sigma
        rank_mu = sum(weights[i] * np.outer(elite_y[i], elite_y[i]) for i in range(m_elite))
        C = (
            (1.0 - c1 - c_mu + (1.0 - h_sigma) * c1 * c_c * (2.0 - c_c)) * C
            + c1 * np.outer(p_c, p_c)
            + c_mu * rank_mu
        )
        sigma *= np.exp((c_sigma / d_sigma) * (norm_p_sigma / expected_norm - 1.0))

    return best_x, best_score

def slsqp_constraints(z, n):
    points = z[:-1].reshape(n, 2)
    p = z[-1]
    d1 = np.linalg.norm(points - ZONE_1, axis=1)
    d2 = np.linalg.norm(points - ZONE_2, axis=1)
    d3 = np.linalg.norm(points - ZONE_3, axis=1)

    constraints = [
        d1 - INNER_RADIUS,
        OUTER_RADIUS - d1,
        d2 - EXCLUSION_RADIUS,
        d3 - EXCLUSION_RADIUS,
    ]
    for i in range(n):
        for j in range(i + 1, n):
            constraints.append([np.linalg.norm(points[i] - points[j]) - p])
    return np.concatenate(constraints)

def refine_layout_slsqp(x0, n, max_steps):
    start_score = layout_score(x0, n)
    if start_score == -np.inf:
```

```
        return x0, -np.inf

    z0 = np.concatenate([x0.copy(), [start_score]])
    result = minimize(
        lambda z: -z[-1],
        z0,
        method="SLSQP",
        bounds=[(1.0, 9.0)] * (2 * n) + [(0.0, 8.0)],
        constraints={"type": "ineq", "fun": lambda z: slsqp_constraints(z, n)},
        options={"maxiter": max_steps, "ftol": 1e-9, "disp": False},
    )

    if not result.success:
        return x0, start_score
    x = result.x[:-1]
    score = layout_score(x, n)
    return (x, score) if score != -np.inf else (x0, start_score)

def optimize_layout(n, restarts, k_max, m, m_elite, seed, greedy_starts, slsqp_steps):
    rng = np.random.default_rng(seed)
    f = lambda x: layout_score(x, n)

    greedy_layouts = greedy_candidate_layouts(rng, n, greedy_starts)
    ranked_starts = sorted(
        ((layout_score(x, n), x) for x in greedy_layouts),
        key=lambda item: item[0],
        reverse=True,
    )

    best_x = None
    best_score = -np.inf
    cma_layouts = []
    for i in range(restarts):
        x0 = ranked_starts[i][1] if i < len(ranked_starts) else None
        x, score = cma_es(f, n, k_max, m, m_elite, rng, x0=x0)
        cma_layouts.append(x)
        if score > best_score:
            best_x, best_score = x, score

    for x0 in cma_layouts:
        x, score = refine_layout_slsqp(x0, n, slsqp_steps)
        if score > best_score:
            best_x, best_score = x, score

    return best_x, best_score
```

A.2 Gaussian Process

```
LENGTH_SCALE = 4.0
NOISE_VARIANCE = 0.02 ** 2

def theoretical_efficiency(p):
    return 1.0 / (1.0 + 1.0 / p)

def squared_exponential_kernel(x1, x2, length_scale=LENGTH_SCALE):
    x1 = np.asarray(x1).reshape(-1, 1)
    x2 = np.asarray(x2).reshape(1, -1)
    return np.exp(-0.5 * ((x1 - x2) / length_scale) ** 2)

def fit_gp_predict(x_train, y_train, x_test):
    mean_train = theoretical_efficiency(x_train)
    mean_test = theoretical_efficiency(x_test)

    k_pred = squared_exponential_kernel(x_test, x_test)
    k_pred_train = squared_exponential_kernel(x_test, x_train)
    k_train = squared_exponential_kernel(x_train, x_train)
    noisy_covariance = k_train + NOISE_VARIANCE * np.eye(len(x_train))

    k_prod_inv = np.linalg.solve(noisy_covariance.T, k_pred_train.T).T
    posterior_mean = mean_test + k_prod_inv @ (y_train - mean_train)
    posterior_cov = k_pred - k_prod_inv @ k_pred_train.T
    posterior_variance = np.maximum(np.diag(posterior_cov), np.finfo(float
        ).eps)

    return posterior_mean, np.sqrt(posterior_variance)

    def build_gp(output_dir):
        x_train = np.array([2.5, 2.5, 4.5, 4.5, 6.5, 6.5, 8.0, 8.0])
        y_train = np.array([0.59, 0.62, 0.82, 0.85, 0.86, 0.88, 0.90,
            0.91])
        x_test = np.linspace(1.0, 10.0, 500)

        mean, std = fit_gp_predict(x_train, y_train, x_test)
        theory = theoretical_efficiency(x_test)

        output_dir.mkdir(parents=True, exist_ok=True)

        fig, ax = plt.subplots(figsize=(8, 5))
        ax.fill_between(
            x_test,
            mean - 1.96 * std,
            mean + 1.96 * std,
            color="tab:blue",
            alpha=0.18,
            label="95%␣confidence␣region",
        )
        ax.plot(x_test, mean, color="tab:blue", linewidth=2.0, label="
            Predicted␣GP␣posterior␣mean")
        ax.plot(x_test, theory, color="black", linestyle="-", linewidth
            =2.0, label="Theoretical␣efficiency␣over␣p")
```

```
        ax.scatter(x_train, y_train, color="tab:red", s=20, zorder=5,
            label="Simulation data")
        ax.set_xlabel("Separation distance p")
        ax.set_ylabel("Diversification efficiency")
        ax.set_title("Gaussian Process Fit to Realized Efficiency")
        ax.set_xlim(1.0, 10.0)
        ax.set_ylim(0.45, 1.05)
        ax.legend()
        fig.tight_layout()
        fig.savefig(output_dir / "gp_efficiency_fit.png", dpi=200)
        plt.close(fig)

        predictions = np.column_stack([x_test, mean, std, mean - 1.96 *
            std, mean + 1.96 * std])
        np.savetxt(
            output_dir / "gp_efficiency_predictions.csv",
            predictions,
            delimiter=",",
            header="p,mean,std,lower_95,upper_95",
            comments="",
        )
```

A.3 Expected Profit

```
from gaussian_process import fit_gp_predict, theoretical_efficiency

CAPITAL_PER_STOCK = 25_000_000.0
ALPHA_PER_STOCK = 0.04
COST_PER_STOCK = 550_000.0
X_TRAIN = np.array([2.5, 2.5, 4.5, 4.5, 6.5, 6.5, 8.0, 8.0])
Y_TRAIN = np.array([0.59, 0.62, 0.82, 0.85, 0.86, 0.88, 0.90, 0.91])

def profit(n, efficiency):
    return n * efficiency * CAPITAL_PER_STOCK * ALPHA_PER_STOCK - n *
        COST_PER_STOCK

def load_layout_results(path):
    data = np.loadtxt(path, delimiter=",")
    data = np.atleast_2d(data)
    return data[:, 0].astype(int), data[:, 1]

def save_profit_outputs(output_dir, image_dir=None):
    base_dir = Path(__file__).resolve().parent
    n_values, distances = load_layout_results(base_dir / "
        portfolio_layout_results.csv")

    gp_mean, gp_std = fit_gp_predict(X_TRAIN, Y_TRAIN, distances)
    gp_lower = gp_mean - 1.96 * gp_std
    gp_upper = gp_mean + 1.96 * gp_std
    theory = theoretical_efficiency(distances)

    theory_profit = profit(n_values, theory)
    mean_profit = profit(n_values, gp_mean)
    lower_profit = profit(n_values, gp_lower)
    upper_profit = profit(n_values, gp_upper)

    output_dir.mkdir(parents=True, exist_ok=True)
    image_dir.mkdir(parents=True, exist_ok=True)

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.fill_between(
        n_values,
        lower_profit / 1e6,
        upper_profit / 1e6,
        color="tab:blue",
        alpha=0.18,
        label="95% confidence region",
    )
    ax.plot(n_values, mean_profit / 1e6, marker="o", color="tab:blue",
        linewidth=2.0, label="GP mean profit")
    ax.plot(n_values, theory_profit / 1e6, marker="s", color="black",
        linewidth=1.8, label="Theoretical profit")
    ax.plot(n_values, lower_profit / 1e6, linestyle="--", color="tab:blue"
        , linewidth=1.2, label="95% lower bound")
    ax.plot(n_values, upper_profit / 1e6, linestyle="--", color="tab:
        orange", linewidth=1.2, label="95% upper bound")
```

```
    ax.set_xlabel("Number␣of␣stocks␣n")
    ax.set_ylabel("Expected␣yearly␣profit␣(million␣USD)")
    ax.set_title("Expected␣Profit␣vs.␣Portfolio␣Size")
    ax.set_xticks(n_values)
    ax.legend()
    fig.tight_layout()

    plot_path = output_dir / "expected_profit_vs_n.png"
    fig.savefig(plot_path, dpi=200)
    fig.savefig(image_dir / "expected_profit_vs_n.png", dpi=200)
    plt.close(fig)
```